

Übungsblatt 3

Mittwoch, 26. November 2025 21:04

2,5/4 Aufgabe 3.1: Echtzeit-Scheduling und Prioritäten

0,5/2 a) Gegeben sei die folgende Menge an Rechenaufträgen mit ihren Planungsdaten:

Prozess	Ankunftszeit	Rechenzeit	Deadline
P_1	0	4	8
P_2	2	3	10
P_3	6	4	14
P_4	6	2	10
P_5	8	5	15

Prüfen Sie, ob ein gültiger Ablaufplan für die gegebenen Aufträge existiert, wenn die Prozesse auf einem Single-Core-System nicht unterbrechend ausgeführt werden!

Hinweis: Deadline meint hier die absolute Deadline, also den entsprechenden Zeitpunkt!

Wir füllen den Plan nur Exemplarisch bei der Problematischen Stelle aus:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a								d							
P2			a								d					
P3							a								d	
P4							a				d					
P5									a							d

Start von P4, P3 und P5 jeweils um 1 Zeiteinheit verschoben. -0,5
P4 startet erst bei 7 statt 6, da P1 + P2 insgesamt für 7 Einheiten rechnen.

Wie man sieht, ist es nicht möglich, dass die Letzten drei Prozesse zu Problemen führen und nicht Abgeschlossen werden können. Selbst wenn man nur die Prozesse P3-5 betrachtet, überschneiden sie sich so, dass P5 nicht abgeschlossen werden kann. Wenn man Den Prozessor auf P5 warten lässt statt P4 zu starten haben wir das Problem, dass P4 nicht abgeschlossen wird.

Einfacher noch kann man sehen, dass die Gesamtrechenzeit länger als die Späteste Deadline ist.

$$4+3+4+2+5 = 18 > 15$$

Damit lassen sich die Prozesse nicht rechtzeitig ausführen.



zu a) Scheduling-Verfahren explizit nennen in Begründung. -1

z. B. Wenn Earliest-Deadline-First nicht funktioniert, dann existiert kein gültiger Ablaufplan.

- 2/2** b) Windows/NT benutzt 16 feste Prioritäten (Prio-Klassen 16 - 31). Geben Sie für folgende Prozesse geeignete Prioritäten an, so daß alle Fristen eingehalten werden! Wie sieht der zugehörige Ablaufplan aus?

Prozess	r_i	t_i	d_i
P_1	0	2	10
P_2	1	4	10
P_3	2	3	15
P_4	8	2	10
P_5	9	4	14

Mit r_i : ready time; t_i : Schätzung für maximale Ausführungszeit; d_i : Deadline.

(2 + 2 Punkte)

Wir erhalten beim Einfügen so das Prozesse P4 und P5 passen, da sie die Engsten Zeitpläne haben und dem Hinzufügen der anderen Prozesse so das alles Passt folgenden Ablaufplan:

Prozess	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
P1	a										d					
P2		a									d					
P3			a													
P4									a		d					
P5										a					d	

a= Ankunft
d= Deadline

Daraus folgt für die Prioritäten:

$P1 < P2$

$P3 < P1, P2$

$P3 < P4, P5$

$P5 < P4$

$\Rightarrow P3 < P1 < P2, P3 < P5 < P4$

Wir setzen um zu verdeutlichen dass P4 und P5 dringend sind:

$P3 < P1 < P2 < P5 < P4$

Obwohl P4 und P5 nicht eine höhere Priority als P1 und P2 haben müssen, da sie sich nicht überschneiden

$P1 = 23$

$P2 = 24$

$P3 = 22$

$P4 = 26$

$P5 = 25$

Wir orientieren uns dabei an der Base Priority Real-Time und haben unterschiedliche Priority Classes

Aufgabe 3.2: Linux CFS-Scheduling

Der Linux Kernel unterstützt seit der Kernel-Version 2.6.24 **Gruppen-basiertes Scheduling**. Beantworten Sie für ein **Einkern** Linux-System mit den folgenden **Gruppen** (/ und tg1) und **Prozessen** (P1, P2 und P3) die folgenden Fragen:

Der Prozess **P3** hat außerdem einen **nice-Wert** von -2, alle anderen Variablen haben ihren **Standardwert**.

- a) Geben Sie die **CFS-Runqueues** (Red-Black-Trees) an, die der Linux-Kernel für die oben genannten Prozesse und Gruppen verwaltet, indem Sie die zugeordneten **Entitäten** angeben.

a)

Der Linux CFS Scheduler legt für jedes Scheduling Entity (SE) eine Runqueue Struktur an.

SE's sind:

Prozesse wie P1, P2, P3

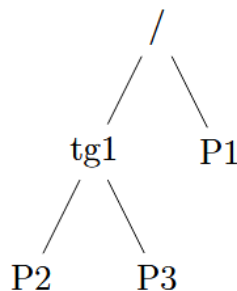
Gruppen wie die Root-Gruppe "/" und die Untergruppe tg1

Also erstellen wir Runqueues für alle SE's.

Als Entitäten erhalten die Queues entweder alle SE's die von der Gruppe abstammen oder "Ø" wenn es sich um einen Prozess handelt.

Runqueue	Entitäten
/	Tg1, P1
tg1	P2, P3
P1	Ø
P2	Ø
P3	Ø

1/1



- b) Diskutieren Sie, zu welchem Zeitpunkt ein Betriebssystem die **vruntime** einer Scheduling-Entität aktualisieren sollte. Überprüfen Sie Ihre Hypothese für den CFS-Scheduler.
- c) Angenommen der Prozess **P3** läuft als erstes auf dem System für **10ms**. Bestimmen Sie die **vruntime** aller betroffenen Scheduling-Entitäten.
- d) Bestimmen Sie die CPU-Zeit der Prozesse P1, P2 und P3 nach einer Sekunde in einem **idealen System**.

(1+1+1+2 Punkte)

b)

Sinnvolle Zeitpunkte können sein:

1. Periodisch: Durch System-Tick sicherstellen, dass lang laufende Prozesse ihre Zeit angerechnet bekommen und ggf. verdrängt werden können.
2. Zustandswechseln: Sobald ein Prozess die CPU freiwillig abgibt oder geweckt wird, damit seine Platzierung im Red-Black-Tree korrekt ist.

Der CFS aktualisiert die vruntime über den function call `update_curr()`, welcher an folgenden Stellen steht:

- `scheduler_tick()` : Periodisch bei jedem Timer-Tick (Interval Abhängig von der HZ - Konfiguration) **richtig** **1/1**
- `dequeue_entity()` / `enqueue_entity()` : Wenn ein Task schlafen geht oder aufwacht
- `put_prev_task()` : Wenn ein Task verdrängt wird oder `yield()` aufruft.

c)

$$vruntime = Laufzeit \times \frac{1024}{weight(nice)}$$

$$weight(-2) = 1586$$

$$vruntime = 10ms \times \frac{1024}{1586} \approx 6,46 ms$$

Also ist die vruntime von P3 nach 10ms 6,46ms **richtig**

Wenn bei einem Prozess die vruntime aktualisiert wird, wird sie es bei alle ihren Vorgängern auch. Das heißt das wenn bei P3 die vruntime bei 6,46ms liegt und da es der erste Prozess ist alle anderen vruntime bei 0ms sind, ist die vruntime von tg1 und "/" auch 6,46ms, also

bei tg1 ist sie allerdings 10ms (standard nice wert)

Runqueue	vruntime
/	6,46 ms
tg1	6,46 ms
P1	0 ms
P2	0 ms
P3	6,46 ms

0.5/1

d)

Time Slice basierend auf den Gewichten in der Hierarchie.

- Gesamtgewicht Root: $1024(P1) + 1024(tg1) = 2048$
- P1: 50%
- tg1: 50%

Zeit für P1: $1000ms \times 0,5 = 500ms$

Zeit für tg1: $1000ms \times 0,5 = 500ms$

die Zeit von tg1 auf SubTask aufteilen.

- Gewicht P2 (Nice 0): **1024**
- Gewicht P3 (Nice -2): **1586**
- Gesamtgewicht in tg1: $1024 + 1586 = 2610$

Zeit für P2:

$$\text{Zeit}_{P2} = 500ms \times \frac{1024}{2610} \approx 500ms \times 0,3923 = 196,17ms$$

Zeit für P3:

$$\text{Zeit}_{P3} = 500ms \times \frac{1586}{2610} \approx 500ms \times 0,6077 = 303,83ms$$

Prozess	CPU-Zeit
P1	500,00 ms
P2	196,17 ms
P3	303,83 ms

2/2

Aufgabe 3.3: 3,5/7

a) 3,5/4

```
#include <stdio.h>
#include "animal.h"
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>
#include <pthread.h>

//static char *words[29];
static double *probability;

void* thread_activities(void *args){
    void **data = (void**)args; // pointer auf void to convert
    stuff
    int j = *(int*) data[0]; // convert void Pointer zu int and
    derefenziere ihn (get the index i)
    char *word = (char*)data[1]; // convert void pointer zu char
    *(probability + j) = animal_probability(word);
    free(data[0]);
    free(args); ✓
    return NULL;
}

int main() {
    char *words[] = {"tiger", "river", "otter", "coffee",
                    "walrus", "router", "yak", "algorithm",
                    "dog", "cat", "math", "bike",
                    "elephant", "table", "guitar", "piano",
                    "lion", "desk", "window", "book",
                    "fish", "banana", "computer", "phone",
                    "car", "take", "monkey", "cash", NULL};

    int total_words = 0;

    // number of total words in words
    for (int i = 0; words[i] != NULL; i++) {
        total_words++;
    }

    probability = (double*) malloc(sizeof(double) *
    (size_t)total_words); // array for probabilities (gets printed
    in the end)
    // size_t converts total_words zu unsigned

    pthread_t thread[total_words]; // array for threads

    int max_threads = 3; ✓
    int active_threads = 0;
    int threads_joined = 0;
```

Fehlende Fehlerbehandlung. -0,5

```

// loop that creates the threads with indexes of the array
and passes them to "thread_activities"
for(int i = 0; i < total_words; i++){

    // arguments for the thread function
    void **args = malloc(sizeof(void*)*2); // array of size 2
for void pointers
    int *index = malloc(sizeof(int));    Siehe oben.

    // pointer to int for index
    *index = i;
    args[0] = index;
    args[1] = words[i];

    // creating one pthread for each loop
    if (pthread_create(&thread[i], NULL, &thread_activities,
args) == 0){
        active_threads++; // if pthread create was successfull ✓
    } else { // if not, wait for the other threads to finish
and then end it
        while(active_threads > 0){
            pthread_join(thread[threads_joined], NULL);
            active_threads--;
            threads_joined++; ✓
        }
        return(EXIT_FAILURE);
    }

    // if 4 threads are running, wait for one to finish bevor
going in the next loop and create another one
    while(active_threads >= max_threads){
        if(pthread_join(thread[threads_joined], NULL) != 0){
            return(EXIT_FAILURE);
        }
        active_threads--; ✓
        threads_joined++;
    }
}

// while there are still threads running, wait for them :
while(active_threads > 0){
    if(pthread_join(thread[threads_joined], NULL) != 0){
        return(EXIT_FAILURE);
    }
    active_threads--; ✓
    threads_joined++;
}

// print the probabilities
for(int i = 0; i < threads_joined; i++){
    printf("%s : %lf\n", words[i],probability[i]); ✓
}

```

```
}  
    // by the way, saying that a yak is less likely to be an  
    animal than a banana, is kinda stupid but i get it  
  
    free(probability);  
    return 0; ✓  
}
```

b & c) 0/3: Keine Bearbeitung.